Selim FARCI

Eugénie MORAIS

1G3 TP3

TP de Numérique : Chronomètre

TP 1

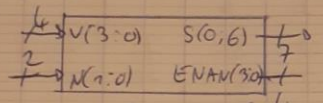
Préparation 1 :

Préparations des TP de numerique.
chronomètre

Préparation 1 :

Le segment i d'un afficheur s'allume si $S(i) = 1$.

Supposons que l'on veut afficher le chiffre 4 sur l'afficheur 2.
On rappel que le module d'affichage Disp1Digit est definit par le bloc suivant :



Avec V désignant les valeur à ~~écrire~~ afficher,
N le code qui numerote l'afficheur selectionné.
S(0:6) qui sont les segments de chaque afficheur.
ENAN(3:0) qui correspond aux 4 signaux envoyés aux 4 anodes.

Dans notre cas, $N = 01$ car c'est l'afficheur 2 qui est selectionné (on commence à 0).
Nous avons $V = 4$ soit en binaire : $V = 0100$
ou $V(3) = 0$, $V(2) = 1$, $V(1) = 0$, $V(0) = 0$.
~~La sortie devra donc et~~
Pour afficher "4", il faut que ~~S(0:6)~~ $S(6) = 1$, $S(5) = 1$, $S(4) = 0$, $S(3) = 0$, $S(2) = 1$, $S(1) = 1$ et $S(0) = 0$ soit $S(0:6) = 1100110$.
Pour selectionner l'afficheur 2, il faut que ~~ENAN(2)~~ $ENAN(3) = 1$, $ENAN(2) = 1$, $ENAN(1) = 0$, $ENAN(0) =$
En effet, $ENAN(i)$ permet de ~~afficher le~~ selectionner l'afficheur

(ENAN las salidas van a ser las bases del transistor PNP).

Tabla de verdad de S6(G) a S0(A)

V_3	V_2	V_1	V_0	S6	S5	S4	S3	S2	S_1	S_0
0	0	0	0	0	1	1	1	1	1	1
0	0	0	1	0	0	0	0	1	1	0
0	0	1	0	1	0	1	1	0	1	1
0	0	1	1	1	0	0	1	1	1	1
0	1	0	0	1	1	0	0	1	1	0
0	1	0	1	1	1	0	1	1	0	1
0	1	1	0	1	1	1	1	1	0	1
0	1	1	1	0	0	0	0	1	1	1
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	0	1	1	1	1
1	0	1	0							
1	0	1	1							
1	1	0	0				>9			
1	1	0	1							
1	1	1	0							
1	1	1	1							

S_5	V_1V_0			
V_3V_2	00	01	11	10
00	1	0	0	0
01	1	1	0	1
11				
10	1	1		

$S_5 = \overline{V_3} \cdot V_1 + V_3 \cdot \overline{V_1} \cdot V_2 + \overline{V_3}\,\overline{V_2}\,\overline{V_1}\,\overline{V_0}$

S_4	V_1V_0			
V_3V_2	00	01	11	10
00	1	0	0	1
01	0	0	0	1
11				
10	1	0		

$V_0 \cdot \overline{V_3} + V_0 \cdot \overline{V_2} + V_0 \cdot \overline{V_1}$

$S_4 = \overline{V_3} \cdot V_1 \cdot \overline{V_0} + \overline{V_3} \cdot \overline{V_2} \cdot \overline{V_0}$

S_3	V_1V_0			
V_3V_2	00	01	11	10
00	1	0	1	1
01	0	1	0	1
11				
10	1	1		

$S_3 = V_3 \cdot \overline{V_2} \cdot \overline{V_1} + \overline{V_3}\,\overline{V_2}\,V_1 + \overline{V_3}\,\overline{V_2}\,\overline{V_0}$

S_2	V_1V_0			
V_3V_2	00	01	11	10
00	1	1	1	0
01	1	1	1	1
11				
10	1	1		

$\cancel{S_2 = \overline{V_3}\,\overline{V_2} \cdot \overline{V_1}\,\overline{V_0}}$

$S_2 = \overline{V_3} + V_3 \cdot \overline{V_2} + \overline{V_3} \cdot \overline{V_2} \cdot V$

$S_2 = \overline{V_3} \cdot \overline{V_2} \cdot \overline{V} + \overline{V_3}\,V_2 + V_3\,\overline{V_2}$

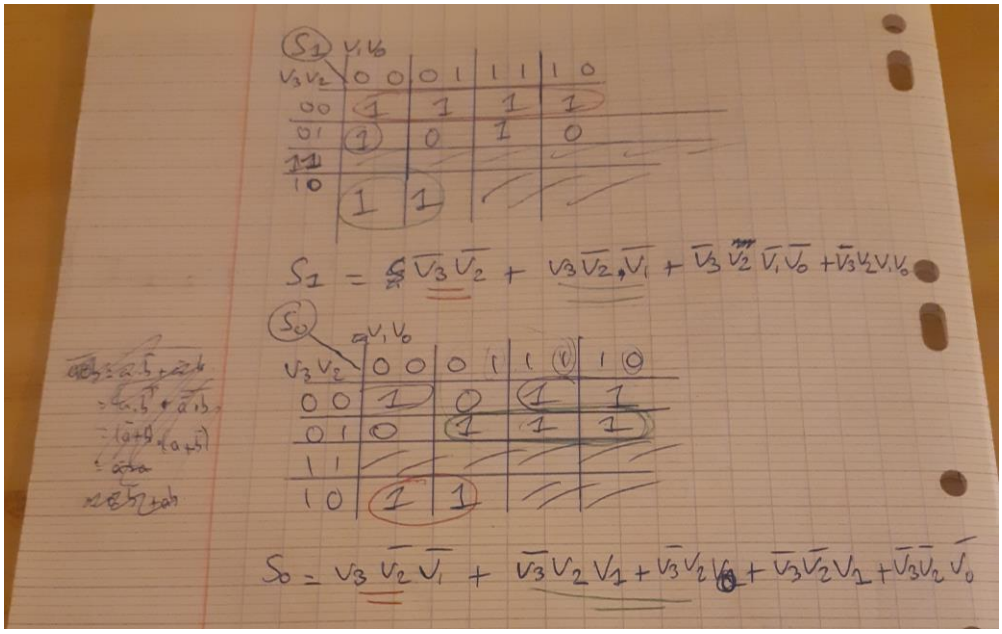
Table de vérité de ENAN(0) à ENAN(3)

~~ENAN(3) ENAN(2) ENAN(1) ENAN(0)~~

N(1)	N(0)	ENAN(3)	ENAN(2)	ENAN(1)	ENAN(0)
0	0	1	1	1	0
0	1	1	1	0	1
1	0	1	0	1	1
1	1	0	1	1	1

Nom dans le doc VHDL	Nom sur la carte	Nom broche du FPGA	type (entrée ou sortie)
V(3)	SW3	W17	Entrée
V(2)	SW2	W16	"
V(1)	SW1	V16	"
V(0)	SW0	V17	"
N(1)			entrée
N(0)			"
S(6)	CG	~~RA~~ U7	Sortie
S(5)	CF	V5	"
S(4)	CE	U5	"
S(3)	CD	V8	"
S(2)	CC	U8	"
S(1)	CB	W6	"
S(0)	CA	W7	"
ENAN(3)	AN3	W4	Sortie
ENAN(2)	AN2	V4	"
ENAN(1)	AN1	U4	"
ENAN(0)	AN0	U2	"

remarque : DP est une cathode LED ~~extrême~~ extrême. les C-lettres sont des diodes électroluminescentes

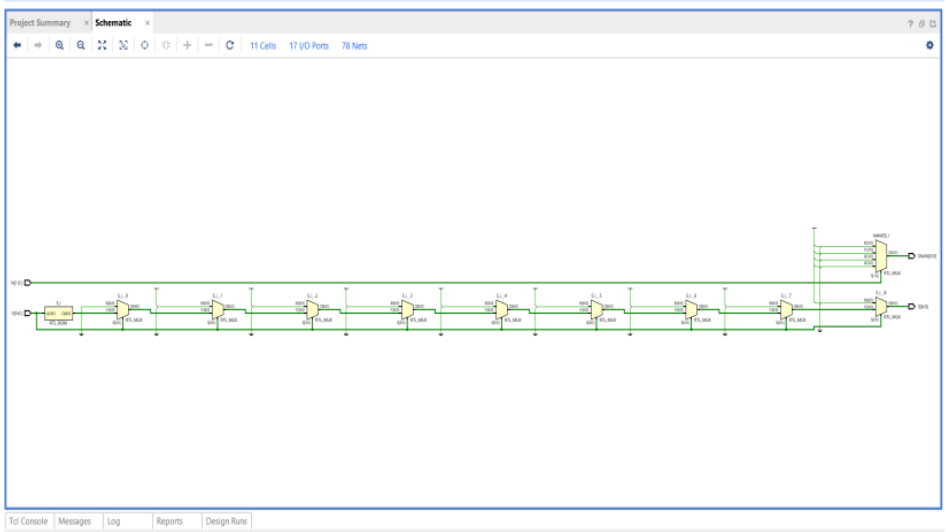


Réalisation 1 :

Codes de S et ENAN ajoutés dans le module “Disp1Digit” :

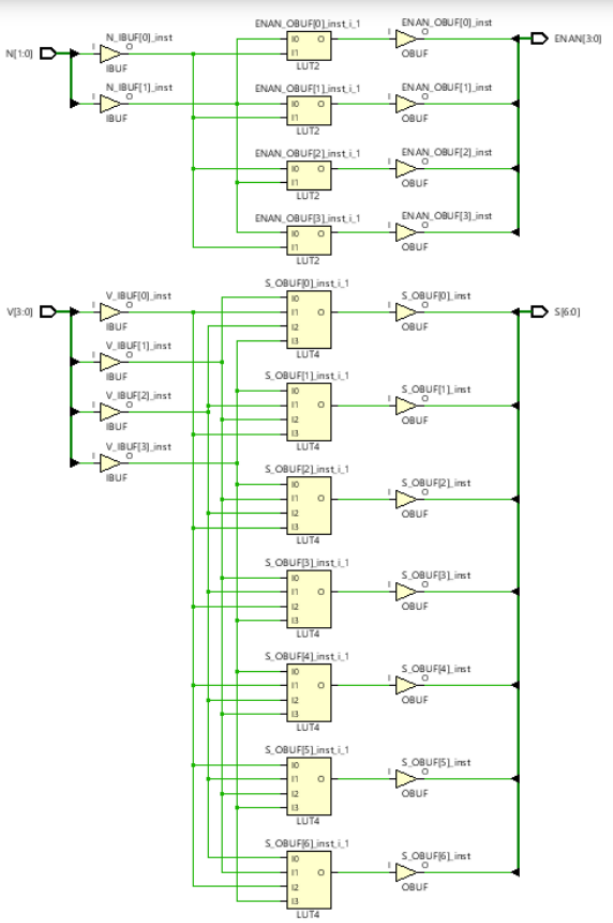
```
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity DisplDigit is
    Port ( V : in STD_LOGIC_VECTOR (3 downto 0);
           N : in STD_LOGIC_VECTOR (1 downto 0);
           S : out STD_LOGIC_VECTOR (6 downto 0);
           ENAN : out STD_LOGIC_VECTOR (3 downto 0));
end DisplDigit;

architecture Behavioral of DisplDigit is

begin
S <=     "1000000" when V="0000" else -- 0
         "1111001" when V="0001" else -- 1
         "0100100" when V="0010" else -- 2
         "0110000" when V="0011" else -- 3
         "0011001" when V="0100" else -- 4
         "0010010" when V="0101" else -- 5
         "0000010" when V="0110" else -- 6
         "1111000" when V="0111" else -- 7
         "0000000" when V="1000" else -- 8
         "0010000" when V="1001" else -- 9
         "1000000" ; -- si V>9, S affiche 0

ENAN <= "1110" when N="00" else -- Cond pour ENAN
        "1101" when N="01" else
        "1011" when N="10" else
        "0111" ;

end Behavioral;
```

Résultat obtenu dans “Schematic”:



Dessin de la structure EN_AN à l'aide de multiplexeurs :

Implémentation 1 :



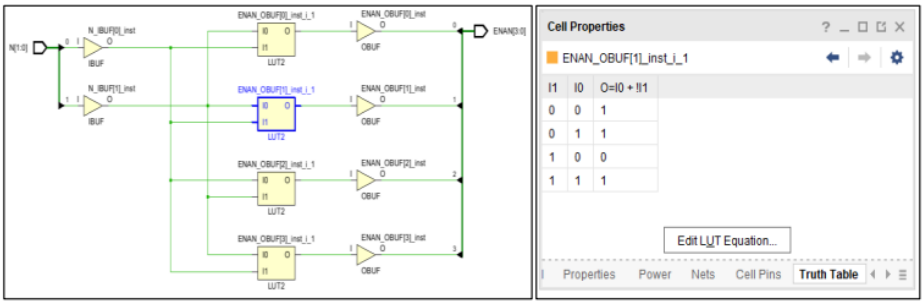
Avec « Primitives », on peut voir qu'il y a 7 LUT4 et 4 LUT2 de mobilisées.

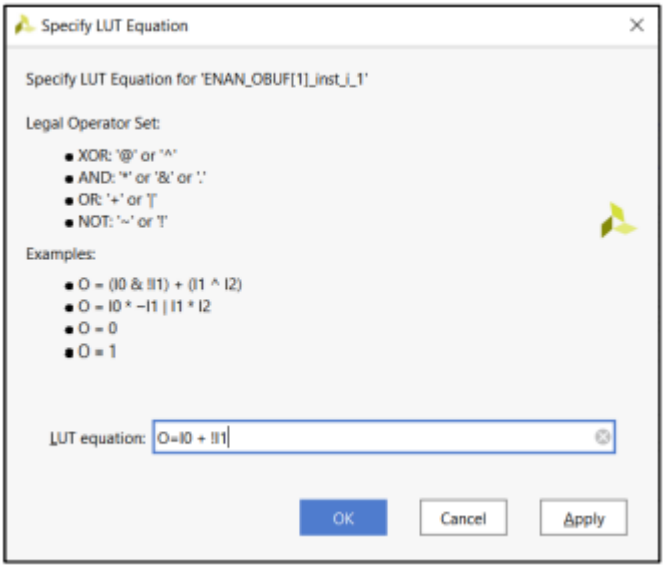
```
7. Primitives
-------------

+----------+------+---------------------+
| Ref Name | Used | Functional Category |
+----------+------+---------------------+
| OBUF     |   11 |                  IO |
| LUT4     |    7 |                 LUT |
| IBUF     |    6 |                  IO |
| LUT2     |    4 |                 LUT |
+----------+------+---------------------+
```

Lorsque l'on parcourt dans le fichier « Utilisation Report », la section primitive synthétise le nombre de LUTs et de buffeurs utilisés dans le montage, on remarque que les chiffres sont conformes.

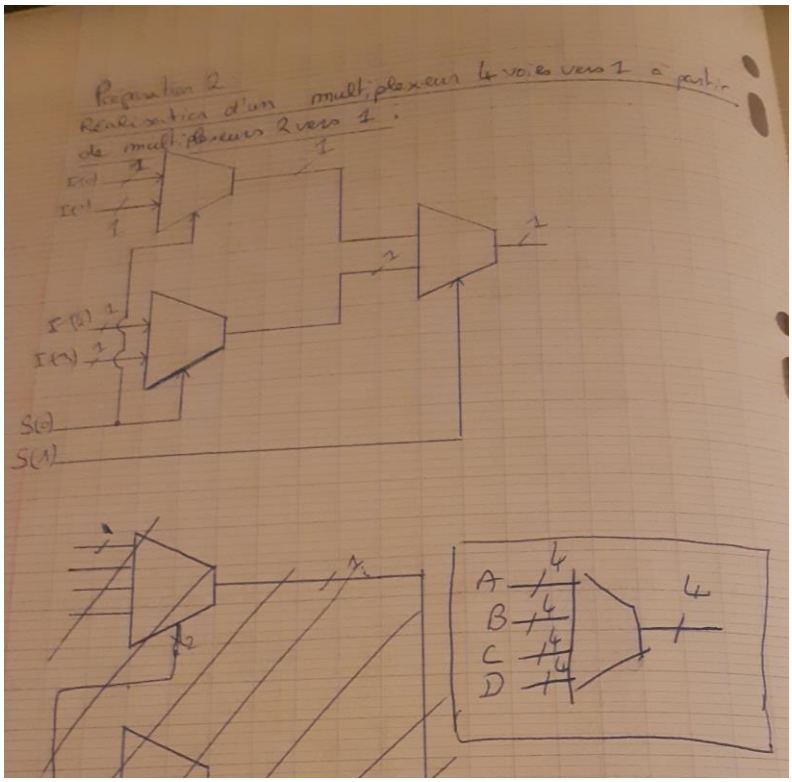
Choix d'une LUT2 :

• Les Buffer permettent l'adaptation de tension entre les entrées analogiques et les niveaux haut/bas de la puce numérique

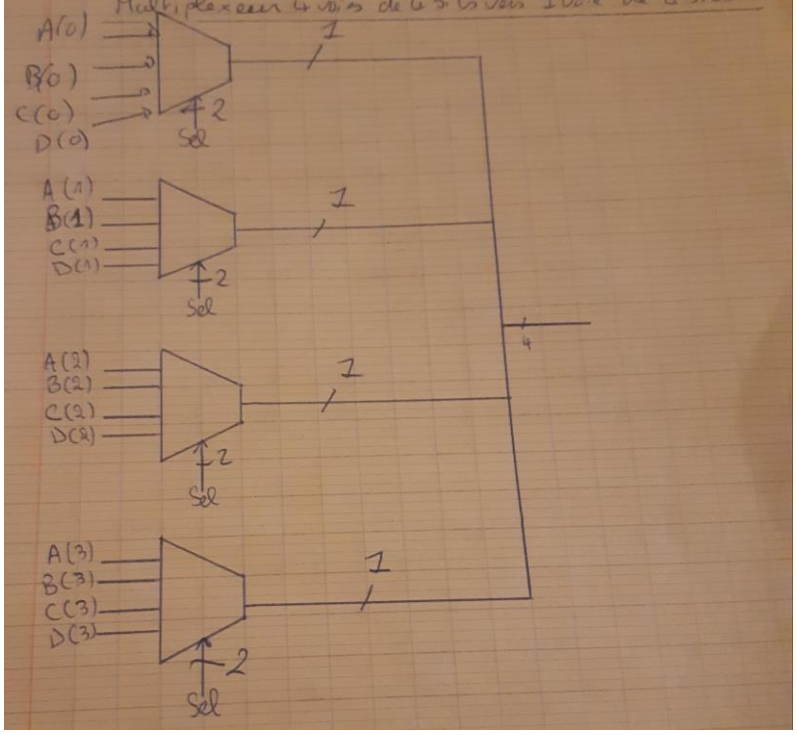




Préparation 2 :



Multiplexeur 4 voies de 4 bits vers 1 voie de 4 bits :



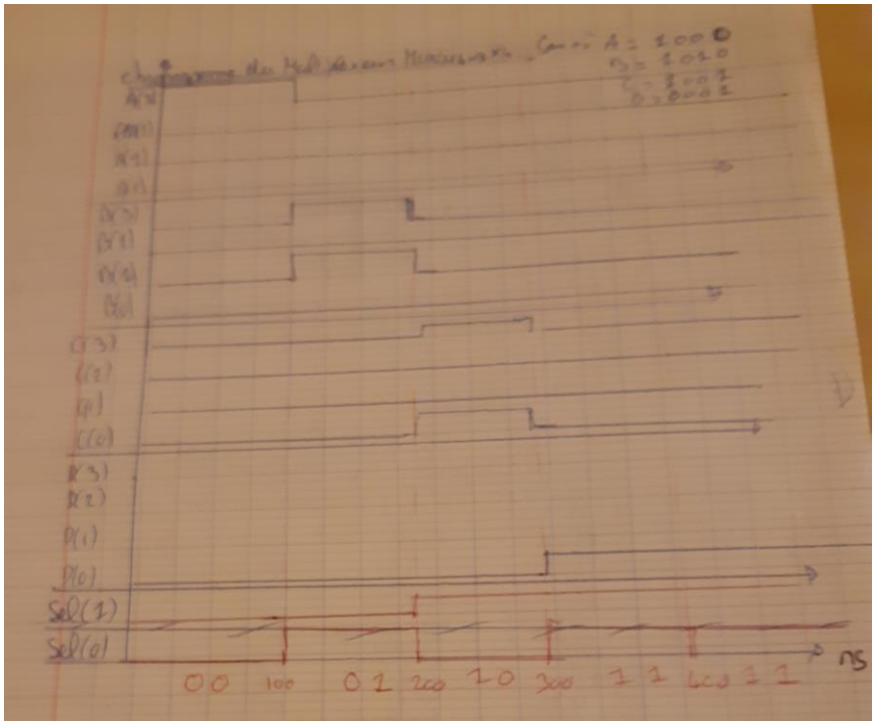
Pour notre module (Multiplexeur 4 voies en un bit vers 1 sur 4 bits), il faut ~~que notre test~~ tester toutes les combinaisons possible de notre multiplexeur. Sachant que notre paramètre sel est définit sur 2 bits, il y a 4 combinaisons à tester : lorsque sel = 00, sel = 01, sel = 10, et sel = 11.

Pour ~~Notre chronogramme devra nous~~ Notre chronogramme devra ~~ressembler~~ être chronogrammes de tests de notre module :

Prenons A = 1000, B = 1010, C = 1001, D = 0001

notre code ici sera du type :

```
sel = 00 ;
wait 100 ns;
sel = 01 ;
wait 100 ns ;
sel = 10 ;
wait 100 ns ;
sel = 11 ;
wait;
```



Réalisation 2 :

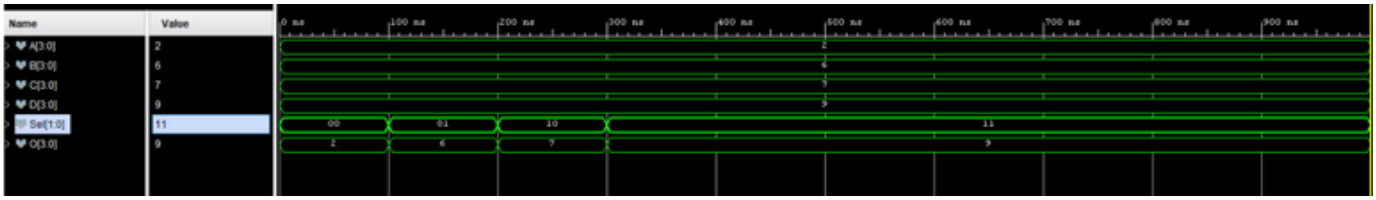
Code VHDL de Disp1of4Digit :

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Displof4Digits is
    Port ( A : in STD_LOGIC_VECTOR (3 downto 0);
           B : in STD_LOGIC_VECTOR (3 downto 0);
           C : in STD_LOGIC_VECTOR (3 downto 0);
           D : in STD_LOGIC_VECTOR (3 downto 0);
           N : in STD_LOGIC_VECTOR (1 downto 0);
           S : out STD_LOGIC_VECTOR (6 downto 0);
           ENAN : out STD_LOGIC_VECTOR (3 downto 0));
end Displof4Digits;

architecture Behavioral of Displof4Digits is
    signal V : STD_LOGIC_VECTOR (3 downto 0);

begin
    u1 : entity work.MUX4x4v1x4
        port map (A=>A, B=>B, C=>C, D=>D, Sel=>N, O=>V);
    u2 : entity work.Displ1Digit
        port map (V=>V, N=>N, S=>S, ENAN => ENAN);



end Behavioral;
```

Simulation 2 :

Code VHDL du “test bench” permettant de simuler le MUX4x4v1x4 :

Résultat de la simulation :



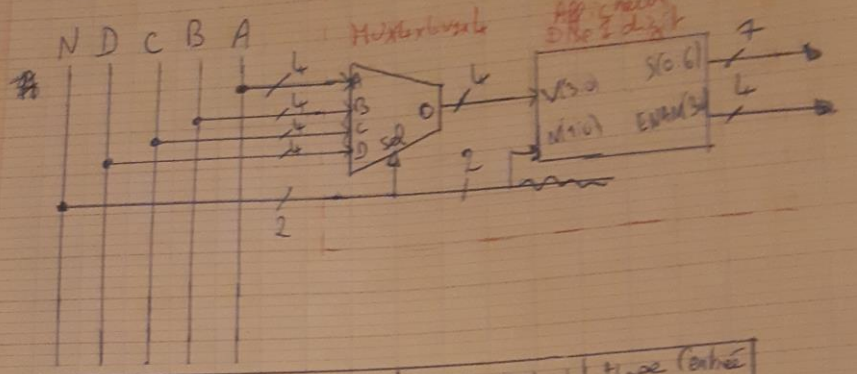
TP2

Préparation 3 :

Préparation 3 :

Disp 1 of 4 Digits :

Disp 1 of 4 Digits



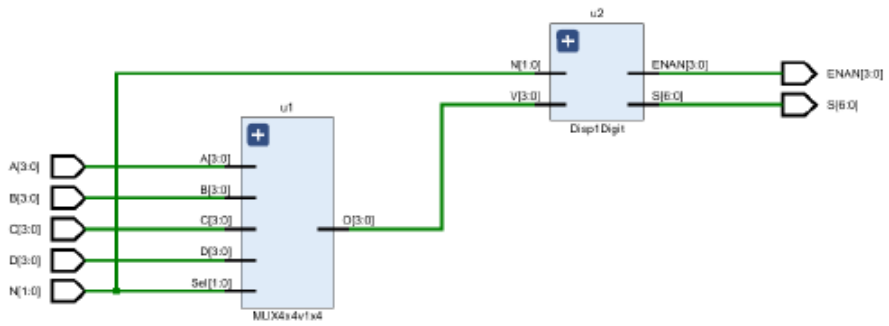
Nom dans la description VHDL (module hiérarchique supérieur)	Nom sur la carte (que vous pouvez lire sur la sérigraphie en TP	Nom sur la broche du FPGA (écrit sur le circuit entre parenthèse)	Type (entrée ou sortie vis-à-vis du FPGA)
D(4)	SW15	R2	Entrée
D(2)	SW14	T1	"
D(1)	SW13	U1	"
D(0)	SW12	W2	"
C(3)	SW11	R3	Entrée
C(2)	SW10	T2	"
C(1)	SW9	T3	"
C(0)	SW8	V2	"
B(3)	SW7	W13	Entrée
B(2)	SW6	W14	"
B(1)	SW5	V15	"
B(0)	SW4	W15	"
A(3)	SW3	W17	Entrée
A(2)	SW2	W16	"
A(1)	SW1	V16	"
A(0)	SW0	U17	"
N(1)		W19	Entrée
N(0)		U18	Entrée
S(6)	GCG	U7	Sortie
S(5)	CF	V5	"
S(4)	CE	U5	"
S(3)	CD	V8	"
S(2)	CC	U8	"
S(1)	CB	W6	"
S(0)	CA	W7	"
ENAN(3)	AN3	W4	Sortie
ENAN(2)	AN2	V4	"
ENAN(1)	AN1	U4	"
ENAN(0)	AN0	U2	"

Réalisation 3 :

Module VHDL Disp1of4Digit créé à l'aide des composants Disp1Digit et MUX4x4v1x4 :

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Displof4Digits is
    Port ( A : in STD_LOGIC_VECTOR (3 downto 0);
           B : in STD_LOGIC_VECTOR (3 downto 0);
           C : in STD_LOGIC_VECTOR (3 downto 0);
           D : in STD_LOGIC_VECTOR (3 downto 0);
           N : in STD_LOGIC_VECTOR (1 downto 0);
           S : out STD_LOGIC_VECTOR (6 downto 0);
           ENAN : out STD_LOGIC_VECTOR (3 downto 0));
end Displof4Digits;

architecture Behavioral of Displof4Digits is
    signal V : STD_LOGIC_VECTOR (3 downto 0);

begin
    u1 : entity work.MUX4x4v1x4
        port map (A=>A, B=>B, C=>C, D=>D, Sel=>N, O=>V);
    u2 : entity work.Disp1Digit
        port map (V=>V, N=>N, S=>S, ENAN => ENAN);



end Behavioral;
```

Schematic : le schéma généré est bien celui auquel on s'attendais (celui trouvé dans la préparation 3).



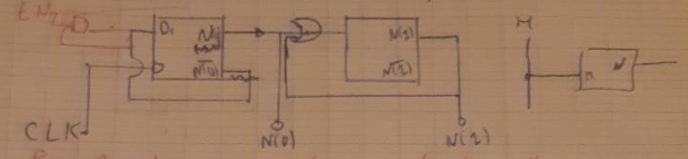
Implémentation 3 :

On copie le fichier de contrainte de Disp1Digit puis on remplace les données par celles trouvées dans la préparation. les afficheurs 7 segments affichent bien ce que l'on désire.

Préparation 4 :

Préparation L :

Synthèse et schéma d'un compteur synchrone cyclique évoluant de 0 à 3 :



Pour le schéma, on ajoute des porte "ET" aux entrées

Schéma du compteur :

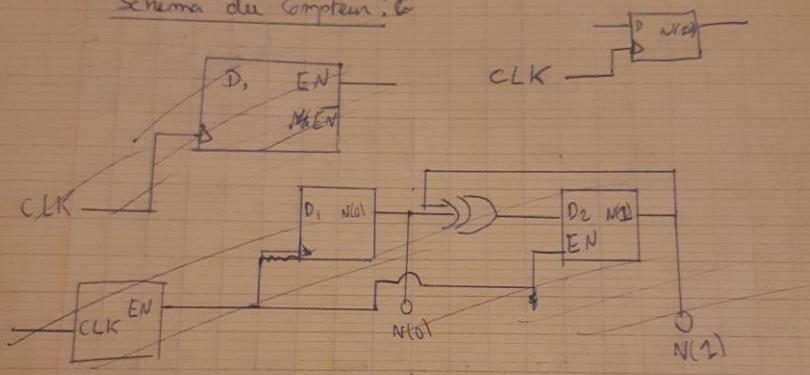


Table de vérité du compteur :

ETAT du compteur	N(1)	N(0)
ETAT 0	0	0
ETAT 1	0	1
ETAT 2	1	0
ETAT 3	1	1

On écrira :

```
if(rising edge (CLK) then
  if(EN='1') then
      M <= M + 1 (binaire sur 2bits)
  else
      M <= M.
```

(EN est juste une autorisation à faire évoluer le compteur

Réalisation 4 :

Module VHDL Cmpt0to3 :

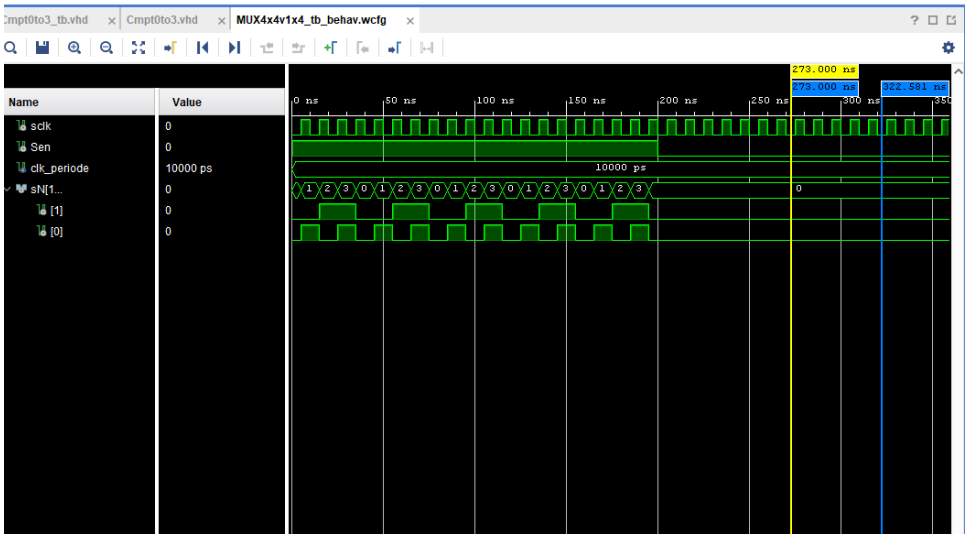
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Cmpt0to3 is
    Port ( CLK : in STD_LOGIC;
           EN : in STD_LOGIC;
           N : out STD_LOGIC_VECTOR (1 downto 0));
end Cmpt0to3;

architecture Behavioral of Cmpt0to3 is
    signal M: unsigned(1 downto 0) := (OTHERS => '0');

begin
process (CLK) begin
    if rising_edge(CLK)then
        if (EN='1')then
            M<= M+1;
        else
            M<=M;
        end if;
     end if;
end process;

N <= STD_LOGIC_VECTOR(M);

end Behavioral;
```

Simulation 4 :

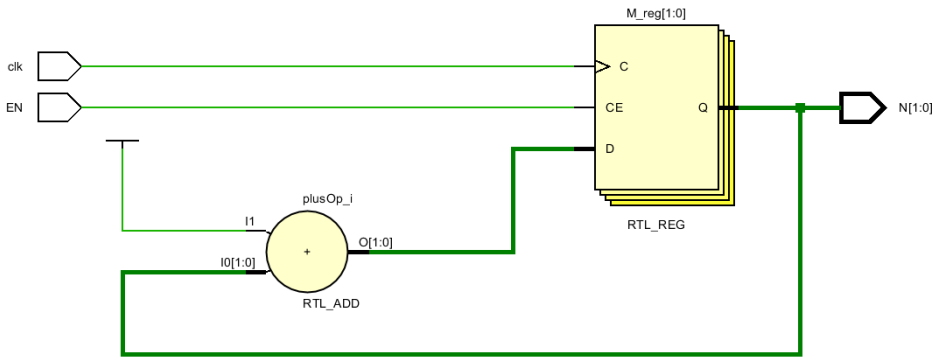
"Test bench" de Cmpt0to3 :

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Cmpt0to3_tb is

end Cmpt0to3_tb;

architecture Behavioral of Cmpt0to3_tb is
    Component Cmpt0to3
    Port ( CLK : in STD_LOGIC;
           EN : in STD_LOGIC;
           N : out STD_LOGIC_VECTOR (1 downto 0));
    end component;
    signal CLK, EN : STD_LOGIC;
    signal N : STD_LOGIC_VECTOR (1 downto 0);
    constant CLK_period : time :=10 ns;

begin
UUT : Cmpt0to3 port map (CLK=>CLK, EN=>EN, N=>N);

CLK_process : process
begin
    CLK<='0';
    wait for CLK_period/2;
    CLK<='1';
    wait for CLK_period/2;
end process;
```

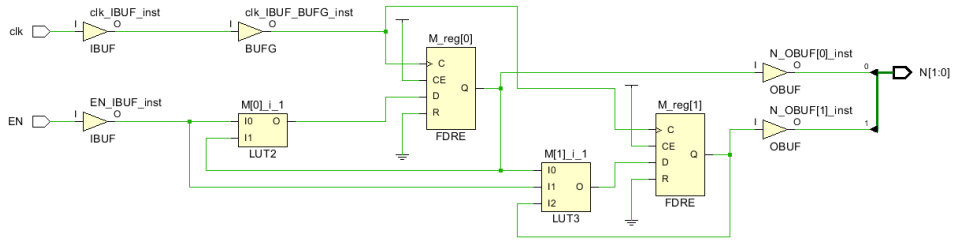
```
process
begin
    EN <= '1';
    wait for 100ns;

    EN <= '0';
    wait for 100ns;

    EN <= '1';
    wait for 100ns;

    wait;
end process;

end Behavioral;
```



Implémentation et analyse 4 :

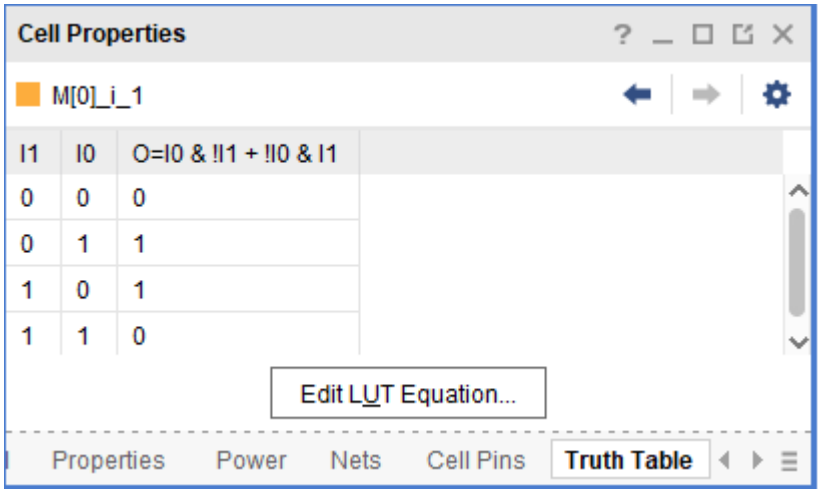


Sur le schéma RTL, on constate qu'il est constitué d'un multiplexeur, un additionneur et 4 régulateurs pour une addition modulo 4 ; le multiplexeur permet de réaliser la fonction « if EN = 0», l'état du signal M est à l'instant présent, la réponse de ce « if » va être envoyé au régulateur (bascule) synchronisé, pour décider l'état futur de M. Si celui-ci est vrai, on garde l'état présent, sinon, on passe par l'additionneur pour incrémenter M de 1



Le schéma généré par Analysis est différent de celui que nous avons réalisé en préparation, car le Analysis n'a pas généré de bascules D, il a ainsi utilisé le composant FDRE qui remplace le fonctionnement de notre bascule.

Tables des vérités des LUTs :

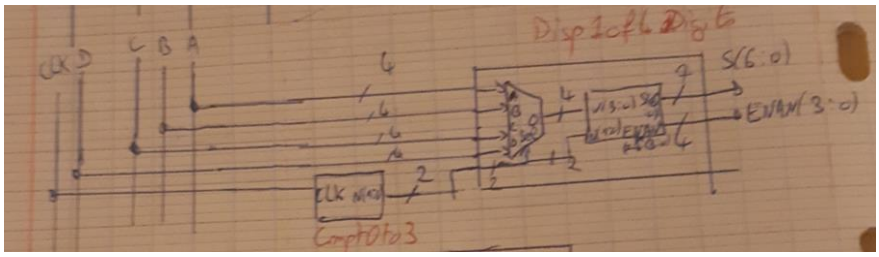


I1	I0	O=I0 & !I1 + !I0 & I1
0	0	0
0	1	1
1	0	1
1	1	0

M[1]_i_1

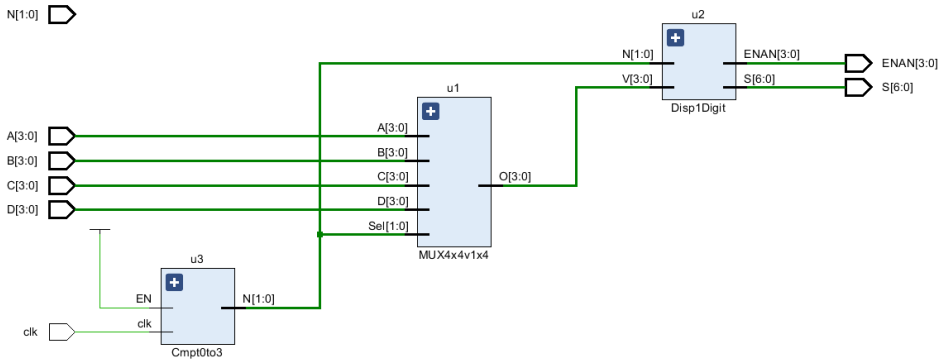
I2	I1	I0	O=I0 & I1 & !I2 + !I1 & I2 + !I0 & I2
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

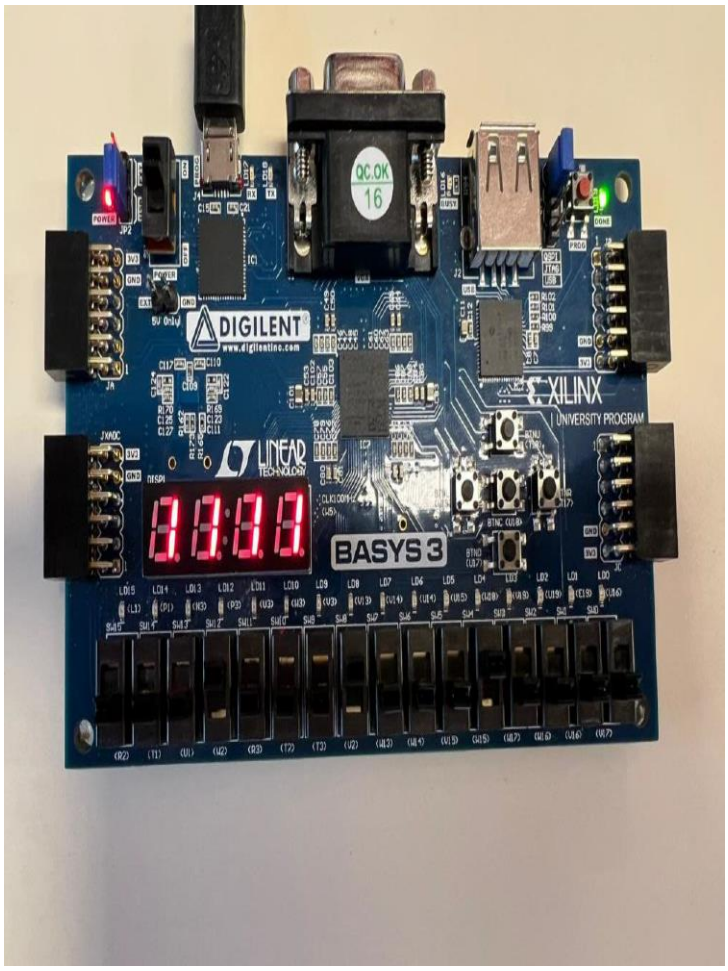
Préparation 5 :



Réalisation 5 :

Module VHDL Disp4D :





Implémentation et mesures 5 :

Si l'on positionne 4 valeurs différentes sur les 4 groupes de switchs, on ne peut pas observer les valeurs sur l'afficheur car les segments sont tous allumés quasiment en même temps en raison de la fréquence trop élevée de l'horloge.

TP 3 :

Préparation 6 :

Préparation 6 :

1 Le chronomètre est trop rapide. Nous allons donc introduire un composant virtuelle "Tick-1 ms" qui va permettre de générer un signal (ici "Tick") qui passe à 1 pendant une période de CLK ($= \frac{1}{100000}$ ms $= 0,00001$ ms).
Soit f' notre notre nouvelle fréquence, et f la fréquence de l'horloge

$f = \frac{1}{10^5} f'$, la fréquence est divisé par $\frac{1}{10^5}$.

On a une division par 10^5, donc il faut compter 10^5 ~~[illegible]~~ front montant.

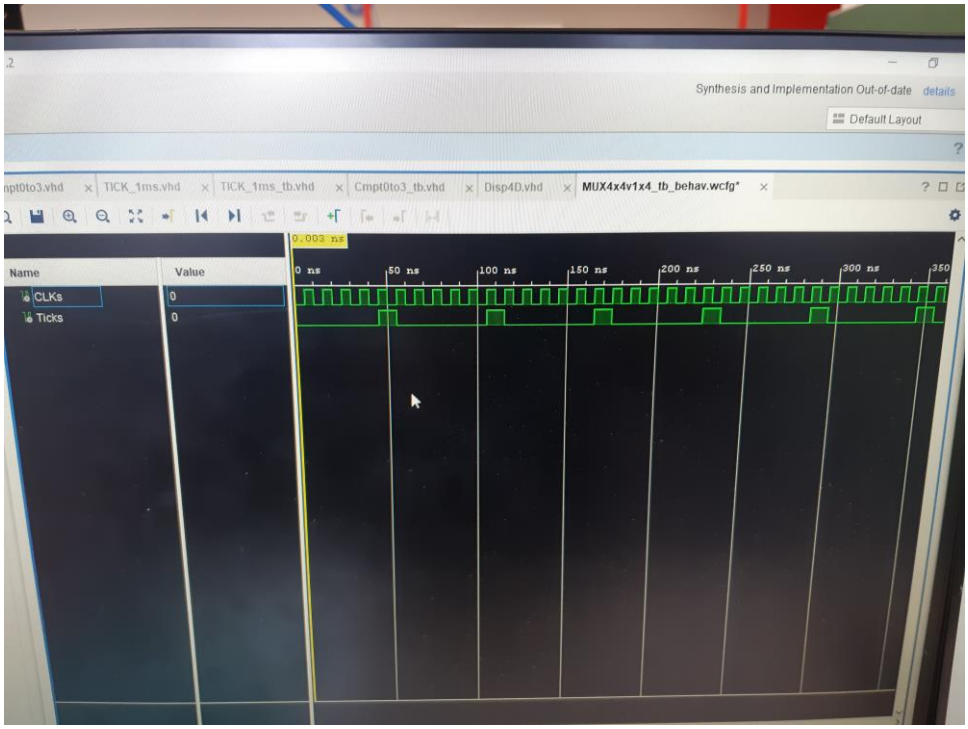
~~Notre [illegible]~~ $= [\log_2(10^5)] = 17$ bascules.

Réalisation 6 :

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;


entity TICK_1ms_tb is
--  Port ( );
end TICK_1ms_tb;

architecture Behavioral of TICK_1ms_tb is
 component TICK_1ms
 Port (    clk : in STD_LOGIC;
            TICK : out STD_LOGIC );
 end component;
 signal sTICK, sclk : STD_LOGIC;
 constant clk_period : time:=10 ns;
begin
UTT : TICK_1ms port map (clk=>sclk, TICK=>sTICK);
    -- Clock process definitions
clk_process :process
    begin
            sclk <= '0';
            wait for clk_period/2;
            sclk <= '1';
            wait for clk_period/2;
    end process;

end Behavioral;
```

Simulation 6 :

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;


entity TICK_1ms_tb is
--  Port ( );
end TICK_1ms_tb;

architecture Behavioral of TICK_1ms_tb is
 component TICK_1ms
 Port (    clk : in STD_LOGIC;
           TICK : out STD_LOGIC );
 end component;
 signal sTICK, sclk : STD_LOGIC;
 constant clk_period : time:=10 ns;
begin
UTT : TICK_1ms port map (clk=>sclk, TICK=>sTICK);
    -- Clock process definitions
clk_process :process
    begin
            sclk <= '0';
            wait for clk_period/2;
            sclk <= '1';
            wait for clk_period/2;
    end process;

end Behavioral;
```



Notre période est de 100 ns ce qui correspond bien à notre attente.

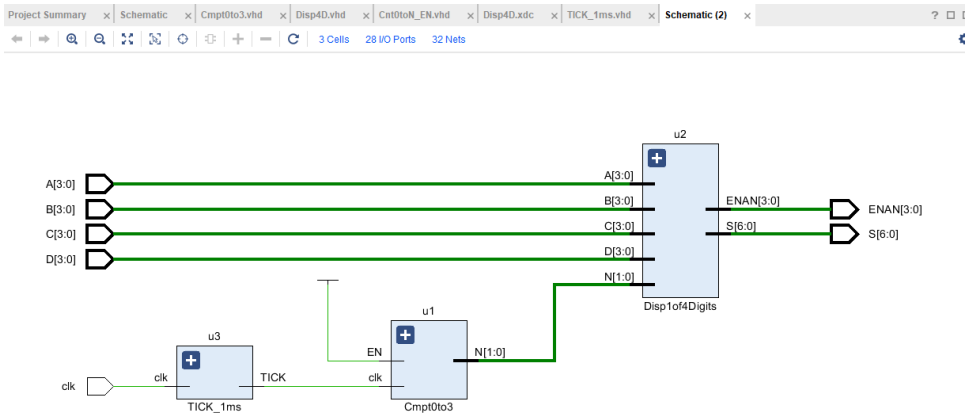
Préparation 8 : Module de Cnt0toN_EN

On réalise à présent un module permettant de commander à un bon rythme l'afficheur 7 segments et dans la suite compter le temps. CLK sera le signal de synchronisation et Tick sera le signal généré. Le code VHDL du module Tick_1ms est présenté ci-dessous :

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;


entity TICK_1ms is
  Port (     clk : in STD_LOGIC;
             TICK : out STD_LOGIC );
end TICK_1ms;

architecture Behavioral of TICK_1ms is
    constant n_div: integer := 5;
    signal Q: integer range 0 to n_div := 0;
begin
    process(clk)begin
        if rising_edge(clk)then
            if(Q = n_div-1)then
            Q <= 0;
            else
            Q <= Q+1;
            end if;
        end if;
    end process;
Tick<= '1' when Q = 0 else '0';


end Behavioral;
```

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;


entity TICK_1ms_tb is
--  Port ( );
end TICK_1ms_tb;

architecture Behavioral of TICK_1ms_tb is
 component TICK_1ms
 Port (     clk : in STD_LOGIC;
             TICK : out STD_LOGIC );
 end component;
 signal sTICK, sclk : STD_LOGIC;
 constant clk_period : time:=10 ns;
begin
UTT : TICK_1ms port map (clk=>sclk, TICK=>sTICK);
    -- Clock process definitions
clk_process :process
    begin
            sclk <= '0';
            wait for clk_period/2;
            sclk <= '1';
            wait for clk_period/2;
    end process;

end Behavioral;
```

Réalisation 7 : On adapte au module Disp4D notre entrée de paramètre (CLK) avec le bon signal (L'horloge à fréquence diminué).

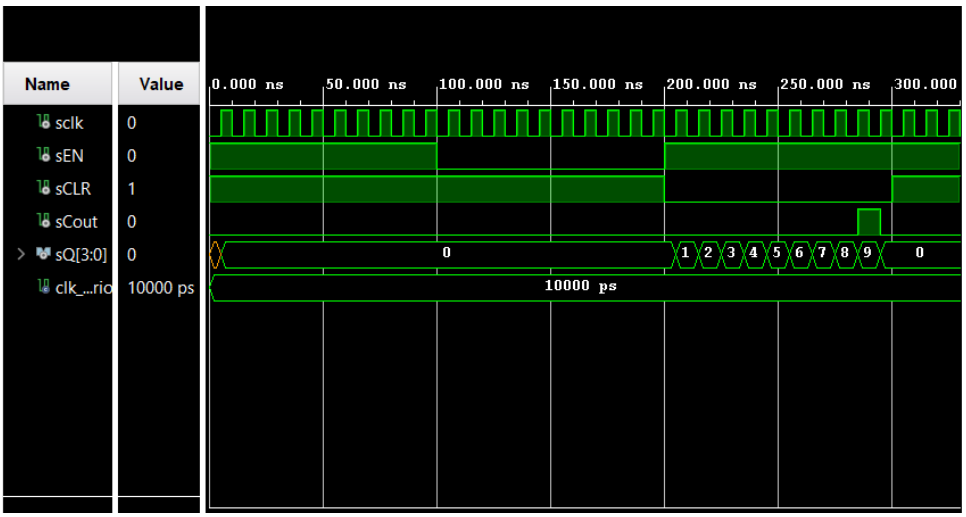


Préparation 8 : même code en VHDL que celui de la réalisation 8.

Réalisation 8 :

```
entity Cnt0toN_EN is
    generic (N : integer :=10);
    Port (      CLK : in STD_LOGIC;
                EN : in STD_LOGIC ;
                CLR : in STD_LOGIC;
                Cout : out STD_LOGIC;
                Q : out STD_LOGIC_VECTOR(3 downto 0));
end Cnt0toN_EN;
architecture Behavioral of Cnt0toN_EN is
signal Q1: STD_LOGIC_VECTOR(3 downto 0);
begin
    process(CLK)
    begin
    if rising_edge(CLK) then
        if CLR = '1' then Q1 <= (others => '0');
        else
            if EN ='1' then
                if (Q1 < N-1) then
                Q1 <= Q1+1;
                else Q1<=(others =>'0');
                end if;
            else
            Q1<= Q1;
            end if;
        end if ;
    end if;
    end process;
    Cout <= '1' when Q1=N-1 else '0';
    Q<=Q1;
end Behavioral;
```

Simulation 8 :

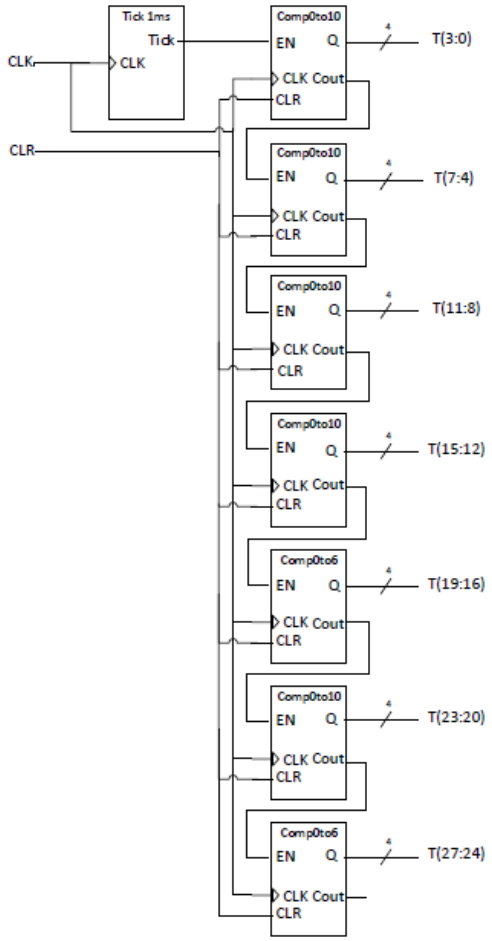


La simulation correspond bien à nos attentes :

Lorsque sCLR = 0, on peux compter et lorsque Cout = 1 alors on arrête de compter.

TP 4 :

Préparation 9 :



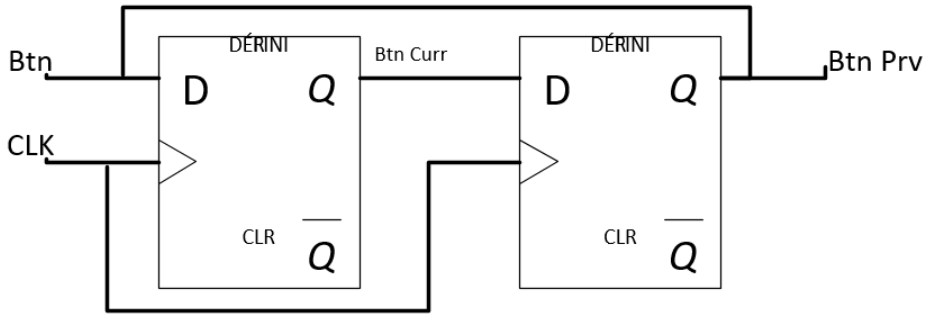
Réalisation 9 : Module VHDL de ChronoEnContinu

L'objectif est maintenant de venir afficher les minutes sur les 2 afficheurs gauches et les secondes sur les 2 à droite

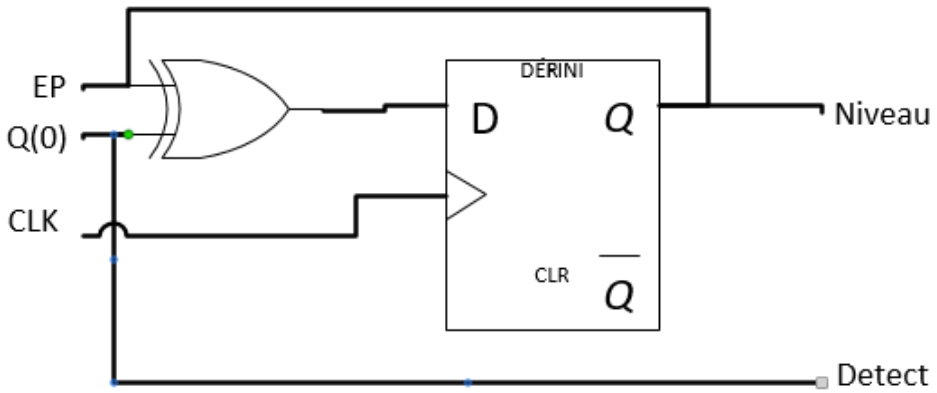
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Chrono is
    Port ( CLK : in STD_LOGIC;
           EN : in STD_LOGIC;
           CLR : in STD_LOGIC;
           T : out STD_LOGIC_VECTOR (27 downto 0));
end Chrono;

architecture Behavioral of Chrono is
    signal Tick : STD_LOGIC;
    signal ENms : STD_LOGIC;
    signal ENcs : STD_LOGIC;
    signal ENds : STD_LOGIC;
    signal ENs : STD_LOGIC;
    signal ENDizs : STD_LOGIC;
    signal ENm : STD_LOGIC;
    signal ENDm : STD_LOGIC;
    signal temp : STD_LOGIC;

begin

    diviseur : entity work.Tick_1ms
        port map (CLK => CLK, Tick => temp);

    ENms <= temp and EN;

    CNTms : entity work.Cnt0toN_EN
        generic map (N=>10)
        port map (CLK => CLK, EN => ENms, CLR => CLR, Q => T(3 downto 0), COUT => ENcs);
    CNTcs : entity work.Cnt0toN_EN
        generic map (N=>10)
        port map (CLK => CLK, EN => ENcs, CLR => CLR, Q => T (7 downto 4), COUT => ENds);
    CNTds : entity work.Cnt0toN_EN
        generic map (N=>10)
        port map (CLK => CLK, EN => ENds, CLR => CLR, Q => T (11 downto 8), COUT => ENs);
    CNTs : entity work.Cnt0toN_EN
        generic map (N=>10)
        port map (CLK => CLK, EN => ENs, CLR => CLR, Q => T (15 downto 12), COUT => ENdizs);
    CNTDizs : entity work.Cnt0toN_EN
        generic map (N=>10)
        port map (CLK => CLK, EN => ENDizs, CLR => CLR, Q => T (19 downto 16), COUT => ENm);
    CNTm : entity work.Cnt0toN_EN
        generic map (N=>10)
        port map (CLK => CLK, EN => ENm, CLR => CLR, Q => T (23 downto 20), COUT => ENDm);
    CNDm : entity work.Cnt0toN_EN
        generic map (N=>10)
        port map (CLK => CLK, EN => ENDm, CLR => CLR, Q => T (27 downto 24));
end Behavioral;
```

Préparation 11 :

A.



C.

Q(1)	Q(0)	BtnCurr	BtnPrev	D(1)	D(0)
0	0	0	X	0	0
0	0	1	1	0	0
0	0	1	0	0	1
0	1	X	X	1	0
1	0	1	X	1	0
1	0	X	1	1	0
1	0	0	0	0	0

D.